

## C-programmering och ATmega 16

C-programmet skrivs med hjälp av Programmers notepad. Programmet VINAVR kompilerar programmet och skapar bl.a. en `-coff` fil och en `.hex` fil. AVR studio används för att ladda ner programmet till labkortet.

Eftersom det skapas många filer vid kompileringen, så är det lämpligt att skapa en katalog för varje projekt. Gör man detta kan källkodsfilen heta `main.c`

Handledningen till lab 3 innehåller anvisningar till hur programmen skrivs.

Alla C-program måste ha funktionen `main( )`.  
Har skriver vi denna funktion som `void main(void)`

```
    {  
    }
```

Void innebär att inget returneras från funktionen, och att funktionen inte heller tar emot något.

Vid C-programmering använde vi alltid `#include<stdio.h>` Filen `stdio.h` innehåller ett antal definitioner som är nödvändiga för att kompilera källkoden.  
Vid programmering av microcontrollern måste vi istället inkludera `avr/io.h`

Vi har tidigare gjort motsvarande sak vid assemblerprogrammering. Där använde vi `.included "m16def.inc"` här använder vi `# include <avr/io.h>`

( För C-programmering av andra typer av microcontroller så måste man använda en `h`-fil som är specifik för just den microcontrollern.)

Vid programskrivning kan vi nu använda alla namn på I/O-protar, exempelvis `DDRB`, `PORTB`, `PINB` osv. Registren `R0 – R31` kommer vi inte åt. Det är kompilatorn som bestämmer användningen av dessa.

Assemblerinstruktioner kan inte heller användas. ( `LDI`, `OUT`, `IN`, `ADD`, `MOV` osv ).  
Däremot så kan man infoga rutiner skrivna i assembler i ett C-program. Det kommer vi att använda i slutet av kursen.

## Exempel

/\* Detta blir en kommentar som sträcker sig över flera rader.

Glöm inte att kommentera \*/

```
#include<avr/io.h>
```

```
void main()
```

```
{
```

```
    DDRB= 0xff;
```

```
    // sätter B till att vara ut-port
```

```
    PORTB =0xFF;
```

```
    //Ett-ställer hela PORTB
```

```
    DDRD =0;
```

```
    // sätter D till inport
```

```
    while(1)
```

```
    // ger en evig slinga
```

```
    {
```

```
        PORTB = PIND;
```

```
        //tilldelar värdet på PORTD till PORTB
```

```
    }
```

```
}
```

Alla portar är på en byte. Därför är det lämpligt att använda datatypen char.

För att slippa problem med positiva och negativa tal skriver vi:

```
unsigned char variabelnamn;
```

Vi kan också använda datatypen int. På samma sätt skriver man

unsigned int. I det här programmet är en int 16 bitar.

```
unsigned char a = 22;
```

Variabeln a får det decimala värdet 22. Åtta bitar totalt, binärt blir det 1 0110.

```
unsigned char b = 0x16;
```

Variabeln b får det hexadecimala värdet 16, ( som är detsamma som 22 decimalt)

```
unsigned char c= 0b10110; och unsigned char c = 0b00010110;
```

Ger ett binärt värde till variabeln c. ( Det är 22 i båda fallen).

## Varning

Unsigned char d =22; ger d det decimala värdet 22.

Men unsigned char d = 022; eller unsigned char d = 00022;

kommer att ge ett oktalt värde till d. Variabeln d får det decimala värdet 18.

Alltså inga inledande nollor på decimaltal!

## Aritmetiska uttryck

|                |   |
|----------------|---|
| Multiplikation | * |
| Division       | / |
| Modulus        | % |
| Addition       | + |
| Subtraktion    | - |

unsigned char y;

|             |            |                         |
|-------------|------------|-------------------------|
| y = 22 + 5; | ger y = 27 |                         |
| y = 22/5;   | ger y = 4  | Decimaldelen försvinner |
| y = 22%5    | ger y = 2  |                         |

## Bitvisa operatorer

|                |    |                          |
|----------------|----|--------------------------|
| Ett komplement | ~  | tilde                    |
| Högerskift     | >> |                          |
| Vänsterskift   | << |                          |
| AND            | &  | Obs endast ett ochtecken |
| OR             |    | Obs endast ett streck    |
| XOR            | ^  |                          |

## Exempel

unsigned char x, y= 0xC9;

x = ~y;                      ger x = 0x36

x = y<<3;                      skifta tre steg åt vänster  
y = 0b 1100 1001  
lägg till tre nollor till höger ? 0b 1100 1001 000  
tag de 8 minst signifikanta bitarna  
0b 0100 1000 = 0x48

x = y>>4;                      Lägg till fyra nollor efter 0b  
0b 0000 1100 1001 Tag de åtta mest signifikanta bitarna  
x = 0x0C

|               |    |                  |
|---------------|----|------------------|
| x = y & 0x3F; | y  | 1100 1001        |
|               | 3F | <u>0011 1111</u> |

x      0000 1001

And bit för bit   x = 0x09:

x = y | 0x21;

|    |                  |
|----|------------------|
| y  | 1100 1001        |
| 21 | <u>0010 0001</u> |
| x  | 1110 1001        |

Or bit för bit   x = 0xE9

x = y ^ 0x21;

|    |                  |
|----|------------------|
| y  | 1100 1001        |
| 21 | <u>0010 0001</u> |
| x  | 1110 1000        |

XOR   x = 0xE8

I C-kursen använde vi AND och OR som logiska operatörer.

unsigned char a = 9;

unsigned char b = 10;

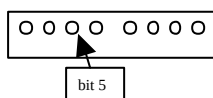
unsigned char z;

z = a && b;      ger z värdet ett , därför båda variablerna är skilda från noll.  
Båda är sanna.

z = a & b;      ger z värdet åtta.

## Maskning

Antag att vi vill ta reda på om bit 5 i PORTB är en etta eller en nolla.



Maskning innebär att man gör en bitvis AND

unsigned char varde;

varde = PORTB & 0x20;

Om bit 5 är en etta så kommer varde att tilldelas talet 0x20, om bit 5 däremot är noll så kommer värde att tilldelas värdet noll.

Övriga bitar kommer inte att påverka resultatet.

Motsatsen till maskning är att sätta en bestämd bit till en etta eller en nolla. Detta utan att påverka de övriga bitarna.

`PORTD = PORTD | 0x04;` Ger en etta på bit 3.

För att nollställa bit 6 kan man använda följande sats:

`PORTD = PORTD & 0xBF;` eller `PORTD = PORTD & (~0x40);`

## Relationsoperatorer

|                          |                    |
|--------------------------|--------------------|
| Lika med                 | <code>==</code>    |
| Inte lika med            | <code>!=</code>    |
| Mindre än                | <code>&lt;</code>  |
| Mindre än eller lika med | <code>&lt;=</code> |
| Större än                | <code>&gt;</code>  |
| Större än eller lika med | <code>&gt;=</code> |

## Control statements

while-loop

```
while( villkor)
{
    .
    .
}
```

Så länge som villkoret är sant kommer satserna inom klamrarna att utföras.

Vi kommer ofta att använda `while(1)`.

Detta är alltid sant, och ger en evig slinga.

for-loop

```
for( i = 0; i<x ; i++)
{
```

```
.  
. }  
}
```

Satserna mellan klamrarna kommer att utföras tills  $i = x$ .

for( ; ; ) kommer att fungera som while(1).

## Arrayer

Man kan skapa en array och sedan läsa ut innehållet med en for-sats.

Exempel:

```
#include<avr/io.h>  
void delay( unsigned char x);  
void main()  
{  
    unsigned char i;  
    DDRB= 0xFF;  
    PORTB =0xFF;  
    DDRD =0x0;  
    unsigned char a[]={1,3,7,0xf,0x1f};  
for( ;;)                                // istället för while(1)  
  
{  
    i=0;  
    for(i=0;i<5;i++)  
    {  
        // för att bryta  
        PORTB =~ a[i];                // inverterat värde skrivs ut  
    }  
}  
}
```

För att avbryta en for-slinga kan man använda break. Vill man bara läsa ut de tre första värdena kan man skriva:

```
if(a[i]==0x0f)  
    break;
```

## If-else-konstruktion

```
#include<avr/io.h>

void main()
{
    DDRD= 0xff;

    DDRB =0x0;
    unsigned char x;
    x= PINB;

    if(x<5)
        PORTD= 0x0f;
    else if (x ==5)
        PORTD = 0xf0;
    else
        PORTD= 0x55;

    stop: goto stop;

}
```

Sista raden. stop: goto stop;

Fungerar som stop: rjmp stop i assemblerprogrammering. Här är det nödvändigt att ha ett avslut på programmet.

Kompilatorn kommer att översätta C-koden till maskinkod. Sista tilldelningssatsen är PORTD = 0X55; Det blir:

```
      E585    LDI    R24,0x55
      BB82    OUT    0x12,R24
      ????
```

Programräknaren pekar alltid på nästa instruktion. Då instruktionen BB82 har laddats in, pekar programräknaren på nästa rad i flashminnet. Om detta tolkas som en instruktion kommer man att få mycket konstiga resultat.

## Switch

Har man många val kan man ersätta if-else if- else med switch( )

### Exempel.

```
// F7_5 switch- case

#include<avr/io.h>

void main()
{
    unsigned char a=0;
    DDRD= 0xff;
    DDRB =0x0;

    a= PINB;
    switch(a)
    {
        case 0: // obs kolon
        case 1:
        case 2:
            PORTD = 0x42;
        case 3:
            PORTD= 0x88;
        default:
            PORTD =0x55;
    }
    stop: goto stop;
}
```

Switch fungerar bara med datatyperna char och int.



## Funktioner

Vi kan använda oss av egendefinierade funktioner. Dessa funktioner måste deklarerars innan de används.

Först en funktion som tar emot ett värde och inte returnerar något värde..

`void delay( unsigned char x )`. En fördröjningsfunktion av samma typ som vi har gjort i assembler.

```
void delay( unsigned char x )
{
    unsigned char a;           // lokal variable
    unsigned int b;
    for(a= 1; a<x ; a++)
    {
        for( b=0 ; b<65535; b++)
            ;                  // semicolon, gör ingenting
    }
}
```

Ett litet exempel med en funktion som returnerar ett värde.

Funktionen tar emot ett värde, skiftar två steg åt vänster och returnerar det nya värdet.

```
unsigned char lasut(unsigned char x)
{
    return x<<2;
}
```

Funktionen anropas enligt:

```
in= PIND;
PORTB = lasut(in);
```

Samma funktion kan man skriva med en pekare .  
Då skickar man adressen till den variabel som man vill ändra.

```
#include<avr/io.h>
void skifta(unsigned char *x);    // funktionen deklareras Observera att
                                  //funktionen inte returnerar något

void main(void)
{
    unsigned char in;
    DDRB= 0xFF;                  // iut-port
    PORTB =0xFF;
    DDRD      =0x0;              //in-port
    while(1)
    {
        in= PIND;
        PORTB = in;
        skifta(&in);              // här anropar man funktionen
        PORTB=in;
    }
}
```

Funktionen definieras.

```
void skifta(unsigned char *x)
{
    *x = *x<<2;
}
```